



A MERN-Stack Property Rental Platform

Technical & Product Overview — Based on Current Codebase

MongoDB · Express · React · Node.js (MERN)

Version 1.0 | 2026

Project Summary

House Rent is a full-stack property rental and listing platform built on the MERN stack (MongoDB, Express, React, Node.js). It allows users to browse and search rental properties, register as a standard user or agent, list and manage their own properties, negotiate through an in-app offer system, and message property owners directly through a built-in chat feature.

Unlike a broad multi-service real-estate suite, this project is a focused, working listing-and-negotiation platform: it covers authentication, property CRUD, client-side search/filtering, offers, and per-property chat — each backed by a real REST API and MongoDB models, and verified directly from the source code.

Architecture at a Glance

Layer	Technology
Frontend	React 19 + Vite, React Router v7, Axios
Backend	Node.js + Express 4
Database	MongoDB (Atlas) via Mongoose 8
Authentication	JWT (jsonwebtoken) + bcryptjs password hashing
Dev tooling	Nodemon (server), ESLint (client)

Repository Structure

Path	Purpose
server/	Express API: routes, controllers, Mongoose models, JWT middleware
server/models/	User, Property, Offer, Chat schemas
server/controllers/	Business logic for auth, property, offer, chat
client/	React app (Vite) – pages, components, API service layer
client/src/pages/	Home, Login, Register, Dashboard, AddProperty, EditProperty, PropertyDetails, Offers, Profile, Chat

Data Models

Four Mongoose schemas define the application's core data. All models use automatic createdAt/updatedAt timestamps.

User

Field	Type / Notes
name, email, password	String, required · password stored as bcrypt hash
role	enum: user agent admin (default: user)

Property

Field	Type / Notes
title, description, location	String, required
price, bedrooms, bathrooms	Number, required
image	String URL (defaults to a placeholder image)
virtualTour	String URL, optional
owner	Reference to User

Offer

Field	Type / Notes
property	Reference to Property, required
buyer / owner	References to User, required
amount	Number, required
status	enum: Pending Accepted Rejected (default: Pending)

Chat

Field	Type / Notes
property	Reference to Property, required
sender / receiver	References to User, required
message	String, required, trimmed

API Reference

All endpoints are prefixed with /api. Routes marked "Auth" require a Bearer JWT issued at login.

Authentication — /api/auth

Method & Path	Auth	Description
POST /register	No	Create a new account (role defaults to "user")
POST /login	No	Validate credentials, return JWT + user object

Properties — /api/property

Method & Path	Auth	Description
GET /	No	List all properties, newest first, with owner info populated
GET /:id	No	Get a single property by ID
POST /	Yes	Create a property, owned by the requesting user
PUT /:id	Yes	Update a property
DELETE /:id	Yes	Delete a property

Offers — /api/offer

Method & Path	Auth	Description
POST /	Yes	Submit an offer on a property
GET /	Yes	Get offers relevant to the logged-in user
PUT /:id	Yes	Update offer status (Accept / Reject)

Chat — /api/chat

Method & Path	Auth	Description
POST /	No*	Send a message tied to a property + sender/receiver
GET /:propertyId	No*	Fetch message history for a property
DELETE /:id	No*	Delete a message

*Chat routes do not currently pass through the auth middleware, unlike Property and Offer routes. Recommended fix before production: add the same auth middleware used elsewhere so only authenticated participants can read or send messages.

Feature Walkthrough

Authentication & Roles

Users register with a name, email, and password (hashed with bcrypt) and are assigned a role of user, agent, or admin. Login returns a signed JWT (7-day expiry) plus the user object, stored client-side and attached to the Navbar to show role-specific labels (👤 User / 🧑 Agent).

Property Listings

Any authenticated user can create, edit, and delete property listings (title, description, location, price, bedrooms, bathrooms, image, and an optional virtual tour link). Listings display the populated owner's name and email.

Search & Filtering

The Home page implements client-side search and filtering: free-text search across title/area/city, city selector, minimum/maximum price, bedroom count, and sort order — all applied to the full property list fetched from the API, with incremental "load more" pagination in the UI.

Offers & Negotiation

Buyers submit an offer amount on a property; the property owner sees incoming offers and can accept or reject them. Offer status moves through Pending → Accepted / Rejected, giving both sides a lightweight negotiation trail.

In-App Chat

Each property has its own message thread between a sender and receiver, letting a prospective renter or buyer message the owner directly without leaving the platform.

Dashboard & Profile

Dashboard lists the properties owned by the logged-in user for quick edit/delete access. Profile shows account details and role.

Security Review & Recommendations

A focused review of the current implementation surfaced a few points worth addressing before any public deployment.

Findings

Area	Observation	Severity
Secrets management	The <code>.env</code> file (MongoDB Atlas credentials + JWT secret) is present in the shipped project archive rather than excluded via <code>.gitignore</code> .	High
Chat routes	POST <code>/</code> , GET <code>/:propertyId</code> , and DELETE <code>/:id</code> on <code>/api/chat</code> are not protected by the auth middleware used elsewhere.	Medium
Password reset	No forgot-password / reset-password flow exists yet.	Low
Server-side validation	Property and Offer fields rely on Mongoose "required" checks only; no dedicated validation/sanitization layer (e.g. Joi, express-validator).	Low
Rate limiting	No rate limiting on auth endpoints (login/register).	Low

Recommended Next Steps

- Rotate the exposed MongoDB Atlas password and JWT secret immediately; move real secrets to environment variables set on the hosting platform, and add `.env` to `.gitignore`.
- Apply the existing auth middleware to all `/api/chat` routes so only the two participants in a conversation can read or send messages.
- Add server-side input validation on property price/bedrooms/bathrooms and offer amount to reject malformed or negative values.
- Add basic rate limiting (e.g. `express-rate-limit`) on `/api/auth/login` and `/register`.
- Consider moving image uploads to a dedicated storage service (e.g. Cloudinary/S3) instead of relying on external image URLs.

Suggested Roadmap

Based on what exists today, these are natural next increments rather than a rebuild — each builds directly on the current models and routes.

Phase	Focus	Example Additions
Now → Hardening	Security & reliability	Fix chat auth gap, rotate secrets, add input validation, error boundaries
Next	Search & discovery	Move search/filtering to the backend (query params + indexes) for scalability
Next	Trust & verification	Email verification, property image uploads, listing approval for agents
Later	Real-time	WebSocket-based live chat instead of request/refresh polling
Later	Payments	Optional deposit/booking payment flow via Stripe or Razorpay

What This Document Changed vs. the Earlier Draft

An earlier overview described features such as MLS integration, a full CRM, market-insight dashboards, virtual-tour media, and e-signature workflows. Those aren't present in this codebase today — this document instead reflects only what the current code implements (auth, listings, search/filter UI, offers, and chat), plus a concrete, code-grounded roadmap for the features that were previously implied.

Running the Project Locally

Prerequisites

- Node.js 18+ and npm
- A MongoDB connection string (local or Atlas)

Server Setup

```
cd server
```

```
npm install
```

Create a .env file with PORT, MONGO_URI, and JWT_SECRET, then run:

```
npm run dev
```

Client Setup

```
cd client
```

```
npm install
```

```
npm run dev
```

Optional: Seed Sample Data

```
npm run seed
```